CALIFORNIA STATE UNIVERSITY
LONG BEACH

CECS 341: Computer Architecture & Organization

Hossein Sayadi, PhD

Department of Computer Engineering & Computer Science
California State University, Long Beach

Fall 2025

MIPS ISA and Assembly Language

Lecture Outline

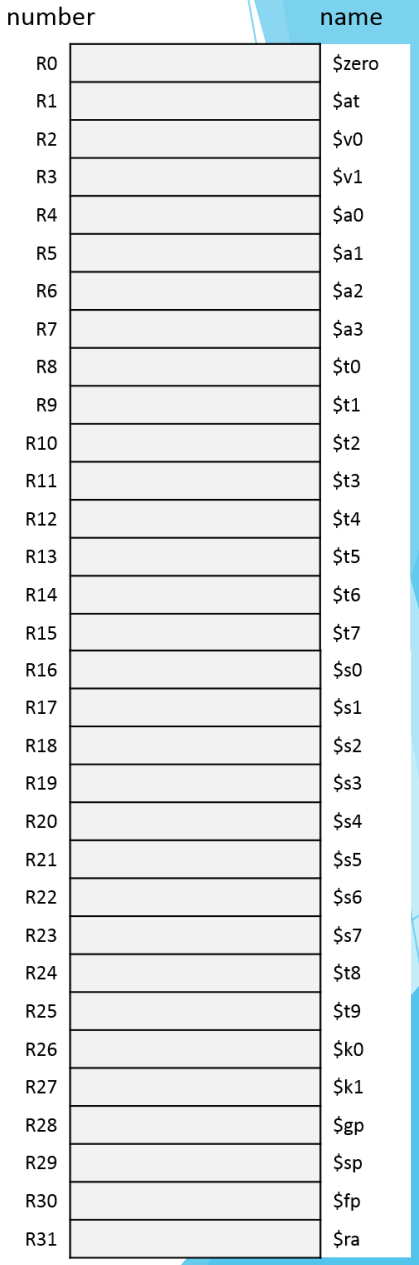
- The MIPS Instruction Set Architecture
  - Review of the MIPS architecture characteristics
- MIPS Assembly Language Instructions
  - Arithmetic and Logical
  - Load and Store
  - Branch, Jump, and Conditional

MIPS Instruction Set

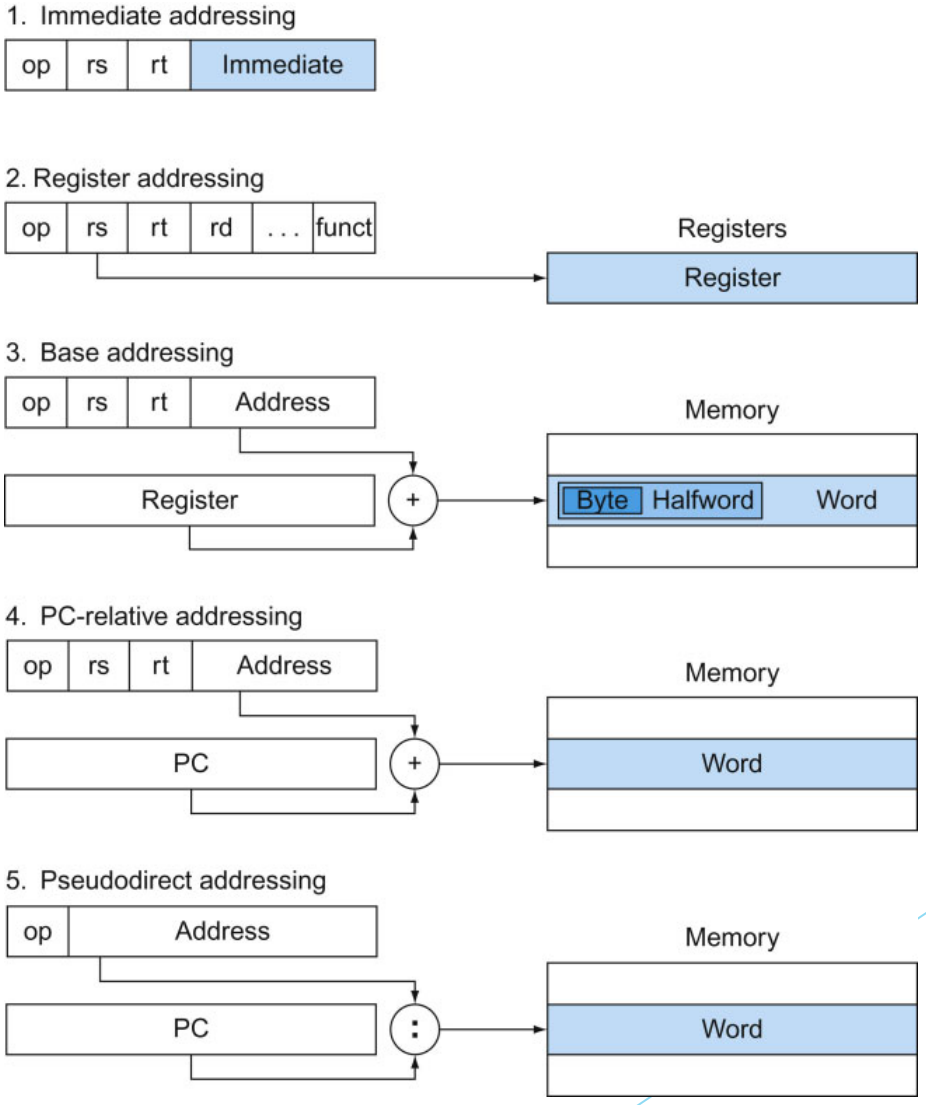
- The MIPS is a ***RISC*** architecture.
- Contains a set of simple instructions.
- Instructions in the Core Instruction Set are classified as
  - Arithmetic
  - Data Transfer
  - Logical
  - Conditional Branch
  - Unconditional Jump
- Other instructions are available for floating point operations.
  - Implemented by the co-processor.

MIPS Register Set

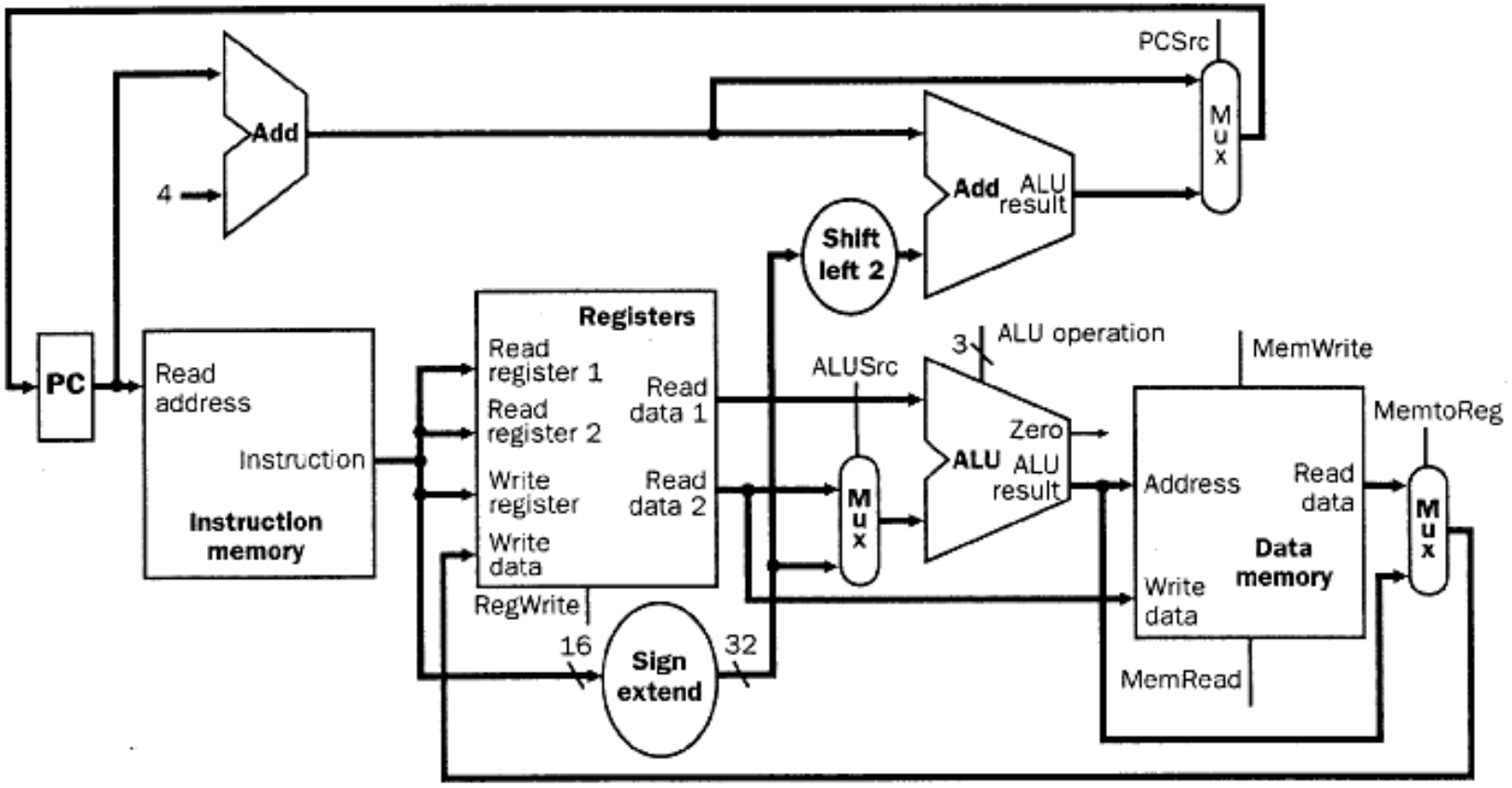
- Consists of **thirty-two** 32-bit registers.
- Most are general-purpose registers.
- Some have special purposes.
  - $zero = contains the value 0.
  - $sp = Stack Pointer
  - $fp = Frame Pointer
  - $ra = Return Address Register
  - others.
- Registers are identified by
  - **Name** in Assembly Language
  - **Number** in Machine Language



MIPS Addressing Modes



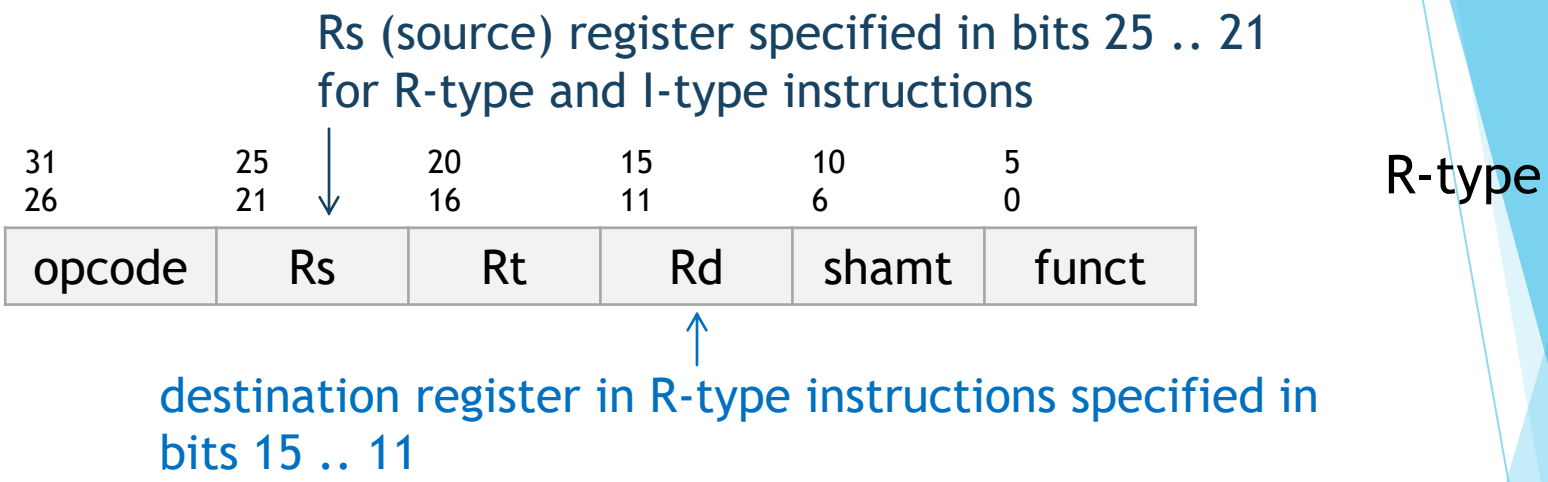
Overview of MIPS Datapath



MIPS Instruction Formats

- The MIPS has **three** instruction formats.
- *Design Principle*: Good design requires good compromise.
  - Ideally, all instructions conform to a single instruction format.
    - Leads to simpler instruction decoding and simpler decode logic.
    - Results in more efficient execution of program instructions.
  - More instruction formats offer greater flexibility.
    - Leads to a greater variety of instructions.
    - Results in an instruction set that provides more functionality.
- The formats are similar.
  - They have common fields, which simplifies instruction decoding.
- The MIPS has **fixed-length** instructions.
  - All instructions are 32-bits long.

MIPS Instruction Formats

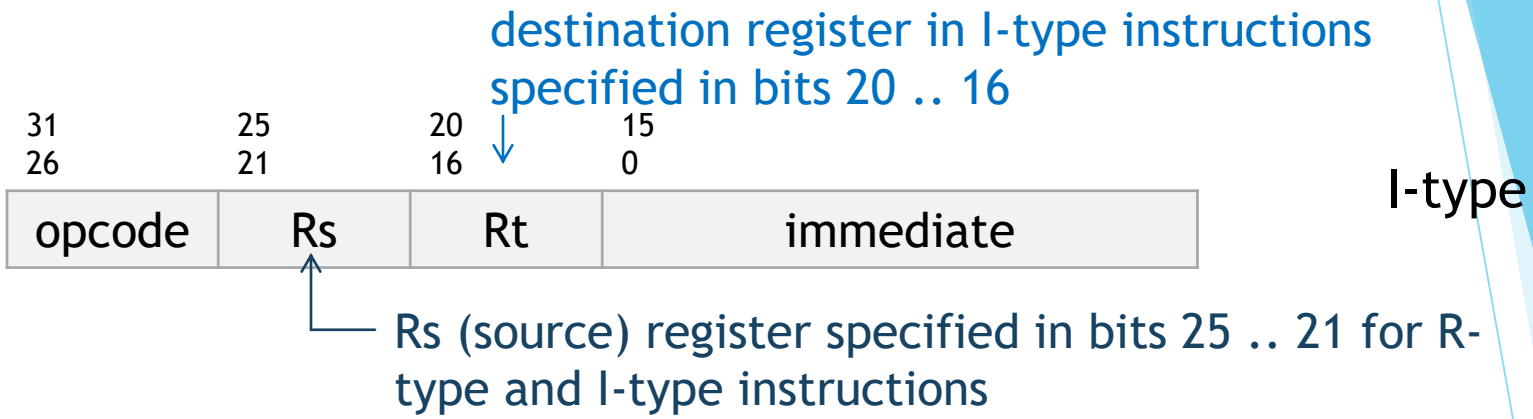


R-type

- Uses three register operands
- R-type format is used by all arithmetic and logical instructions

All MIPS machine language instructions are 32-bits long.

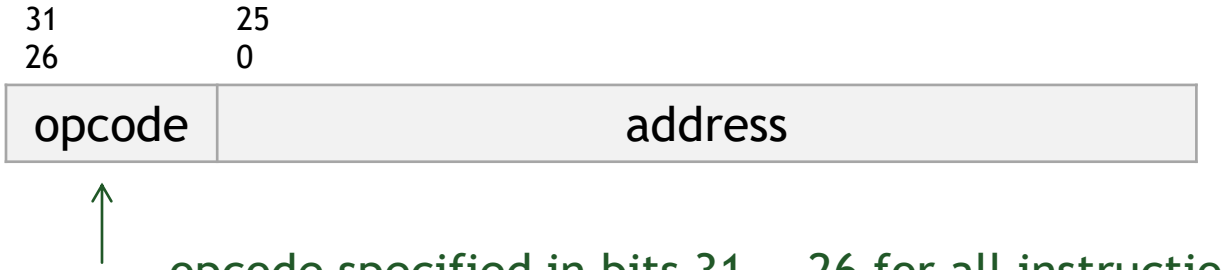
MIPS Instruction Formats



- Uses two register operands and an address/immediate value
- I-type format is used by load and store instructions
- I-type format is used by arithmetic and logical instructions with a constant

All MIPS machine language instructions are 32-bits long.

MIPS Instruction Formats



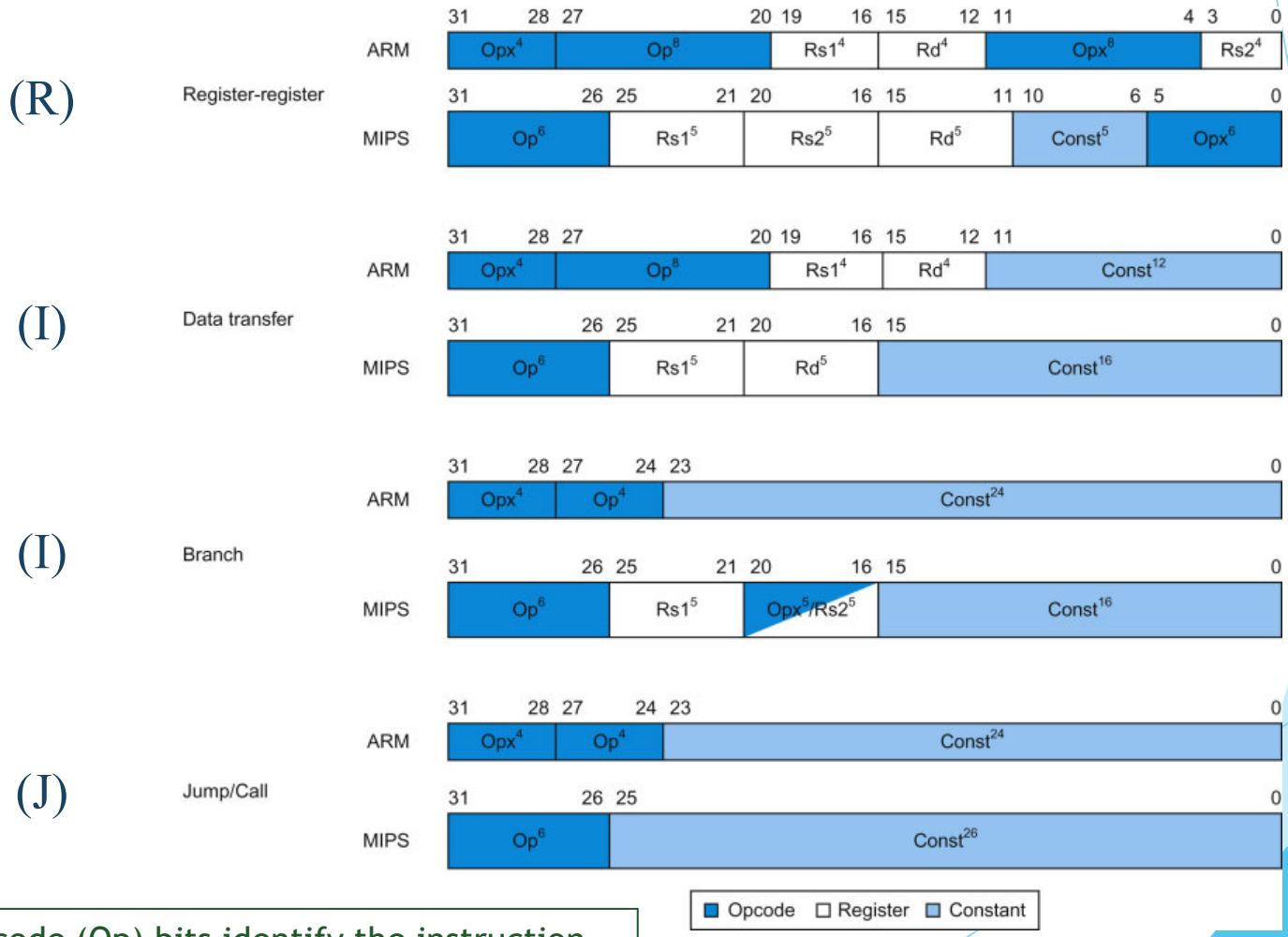
J-type

opcode specified in bits 31 .. 26 for all instructions

- Contains a jump address bits 25 .. 0
- J-type format is used by jump instructions

All MIPS machine language instructions are 32-bits long.

MIPS (and ARM) Instruction Formats



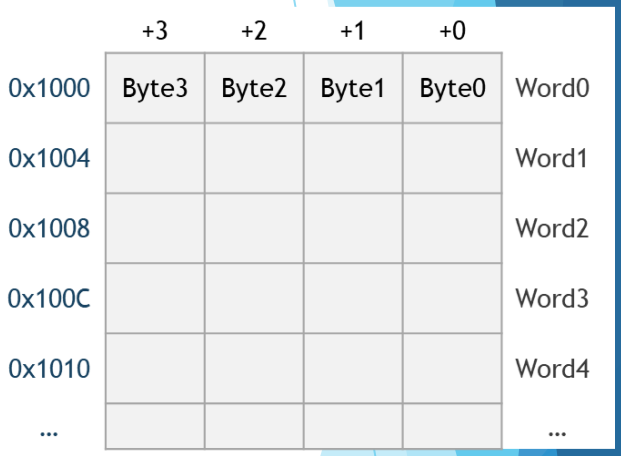
Opcode (Op) bits identify the instruction.

MIPS Data, Address, and Endian-ness

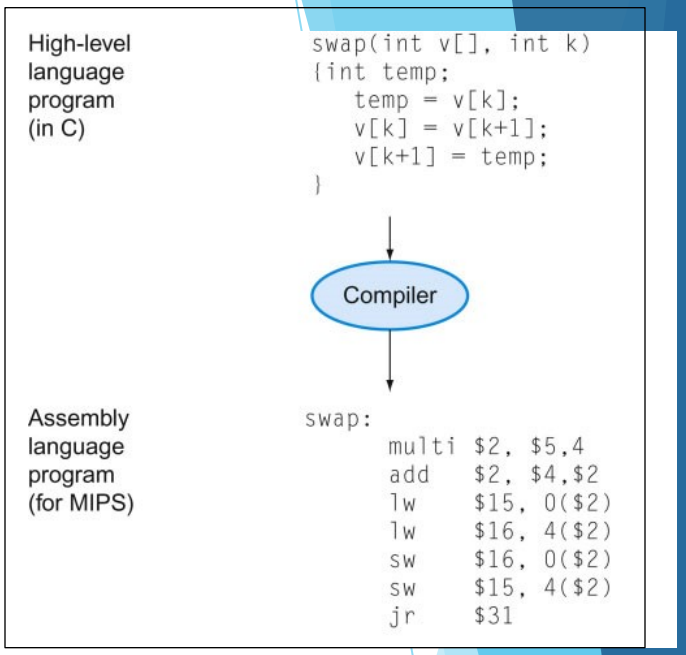
- MIPS Data Width = 32 bits
  - word = 32 bits = 4 bytes
  - Data can be transferred in bytes, half-words, and words.
- MIPS Address Width = 32 bits
  - Can address 4 Gigabytes of memory.
  - Memory is byte-addressable.
- The MIPS is a Bi-endian machine.
  - It can be configured either in Big-endian or Little-endian mode.
  - It can be configured to use either byte ordering.

MIPS Memory Addressing

- The MIPS is a byte-addressable processor.
- Each byte has a unique address.
  - Bytes are located at sequential addresses.
- Each word must be located at a word-boundary.
  - Since a word is composed of four bytes,
  - word addresses must be divisible by four.
  - To point to the address of the next word of memory add 4



Assembly Language



- *Low-level* programming language.
  - "close to" the hardware.
  - Architecture dependent.
- Processor specific.
  - AL instructions are specified in the ISA.
  - Cannot be executed on a different processor.
  - HLL programs must be compiled for a specific processor.
- Compiler translates high-level language instructions into assembly language instructions.
  - Each HLL instruction corresponds to **one or more** AL instructions.
- Human-readable form (of the low-level instructions)
  - Uses mnemonics and names, rather than 0's and 1's.

MIPS Assembly Language Instructions

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	`add $s1,$s2,$s3`	$s1 = $s2 + $s3	Three register operands
	subtract	`sub $s1,$s2,$s3`	$s1 = $s2 – $s3	Three register operands
	add immediate	`addi $s1,$s2,20`	$s1 = $s2 + 20	Used to add constants
Data transfer	load word	`lw $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Word from memory to register
	store word	`sw $s1,20($s2)`	Memory[$s2 + 20] = $s1	Word from register to memory
	load half	`lh $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Halfword memory to register
	load half unsigned	`lhu $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Halfword memory to register
	store half	`sh $s1,20($s2)`	Memory[$s2 + 20] = $s1	Halfword register to memory
	load byte	`lb $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Byte from memory to register
	load byte unsigned	`lbu $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Byte from memory to register
	store byte	`sb $s1,20($s2)`	Memory[$s2 + 20] = $s1	Byte from register to memory
	load linked word	`ll $s1,20($s2)`	$s1 = Memory[$s2 + 20]	Load word as 1st half of atomic swap
	store condition. word	`sc $s1,20($s2)`	Memory[$s2+20]=$s1;$s1=0 or 1	Store word as 2nd half of atomic swap
	load upper immed.	`lui $s1,20`	$s1 = 20 * 2^{16}	Loads constant in upper 16 bits
Logical	and	`and $s1,$s2,$s3`	$s1 = $s2 & $s3	Three reg. operands; bit-by-bit AND
	or	`or $s1,$s2,$s3`	$s1 = $s2 | $s3	Three reg. operands; bit-by-bit OR
	nor	`nor $s1,$s2,$s3`	$s1 = ~ ($s2 | $s3)	Three reg. operands; bit-by-bit NOR
	and immediate	`andi $s1,$s2,20`	$s1 = $s2 & 20	Bit-by-bit AND reg with constant
	or immediate	`ori $s1,$s2,20`	$s1 = $s2 | 20	Bit-by-bit OR reg with constant
	shift left logical	`sll $s1,$s2,10`	$s1 = $s2 << 10	Shift left by constant
	shift right logical	`srl $s1,$s2,10`	$s1 = $s2 >> 10	Shift right by constant
Conditional branch	branch on equal	`beq $s1,$s2,25`	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	`bne $s1,$s2,25`	if ($s1!= $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	`slt $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for `beq`, `bne`
	set on less than unsigned	`sltu $s1,$s2,$s3`	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than unsigned
	set less than immediate	`slti $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant
	set less than immediate unsigned	`sltiu $s1,$s2,20`	if ($s2 < 20) $s1 = 1; else $s1 = 0	Compare less than constant unsigned
Unconditional jump	jump	`j 2500`	go to 10000	Jump to target address
	jump register	`jr $ra`	go to $ra	For switch, procedure return
	jump and link	`jal 2500`	$ra = PC + 4; go to 10000	For procedure call

MIPS AL Instruction Syntax

```
next:   addi $t1, $t1, 1   # increment
```

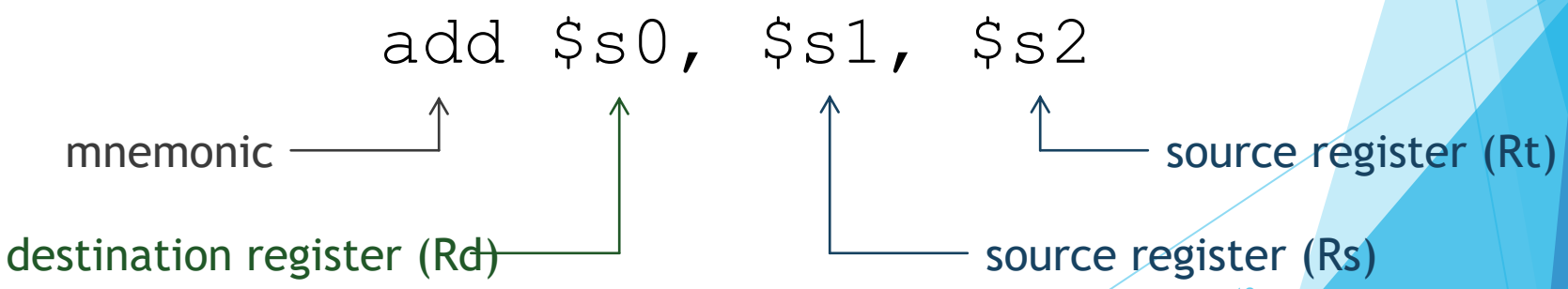


- Label: identifies an assembly language instruction.
  - Represents the address (in program memory) of the instruction.
  - Can be used in another instruction to refer to the labeled instruction.
- Instruction: a MIPS assembly language instruction
- Comment: text to describe the purpose of the instruction.
  - Begins with the # symbol
  - Ends at the end of the line

MIPS Arithmetic Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
add	add Rd, Rs, Rt	R[Rd] = R[Rs] + R[Rt]	R
subtract	sub Rd, Rs, Rt	R[Rd] = R[Rs] – R[Rt]	R
add immediate	addi Rt, Rs, immediate	R[Rt] = R[Rs] + immediate	I

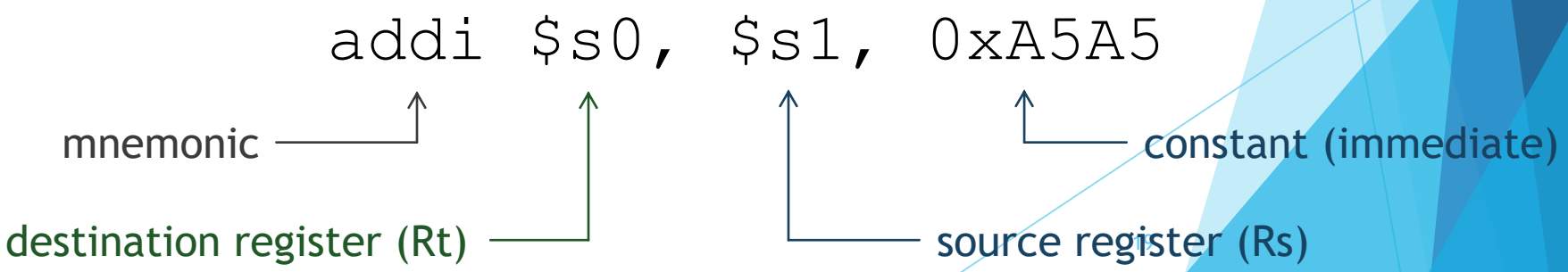
- Arithmetic instructions use register and immediate operands.
  - Cannot operate (directly) on memory operands.
  - Memory operands must first be moved into registers.



MIPS Arithmetic Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
add	add Rd, Rs, Rt	R[Rd] = R[Rs] + R[Rt]	R
subtract	sub Rd, Rs, Rt	R[Rd] = R[Rs] - R[Rt]	R
add immediate	addi Rt, Rs, immediate	R[Rt] = R[Rs] + immediate	I

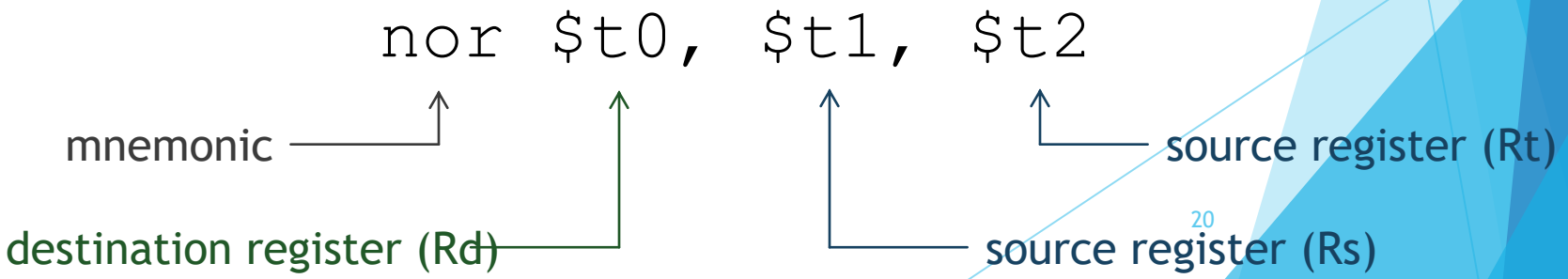
- Arithmetic instructions use register and immediate operands.
  - Cannot operate (directly) on memory operands.
  - Memory operands must first be moved into registers.



MIPS Logical Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
and	and Rd, Rs, Rt	R[Rd] = R[Rs] & R[Rt]	R
or	or Rd, Rs, Rt	R[Rd] = R[Rs] | R[Rt]	R
nor	nor Rd, Rs, Rt	R[Rd] = ~(R[Rs] | R[Rt])	R
and immediate	andi Rt, Rs, immediate	R[Rt] = R[Rs] & immediate	I
or immediate	ori Rt, Rs, immediate	R[Rt] = R[Rs] | immediate	I

- Logical instructions use register and immediate operands.
  - Cannot operate (directly) on memory operands.



MIPS Logical Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
and	and Rd, Rs, Rt	R[Rd] = R[Rs] & R[Rt]	R
or	or Rd, Rs, Rt	R[Rd] = R[Rs] | R[Rt]	R
nor	nor Rd, Rs, Rt	R[Rd] = ~(R[Rs] | R[Rt])	R
and immediate	andi Rt, Rs, immediate	R[Rt] = R[Rs] & immediate	I
or immediate	ori Rt, Rs, immediate	R[Rt] = R[Rs] | immediate	I

- Logical instructions use register and immediate operands.
  - Cannot operate (directly) on memory operands.

```
andi $t0, $t1, 0x5A5A
```

mnemonic

destination register (Rt)

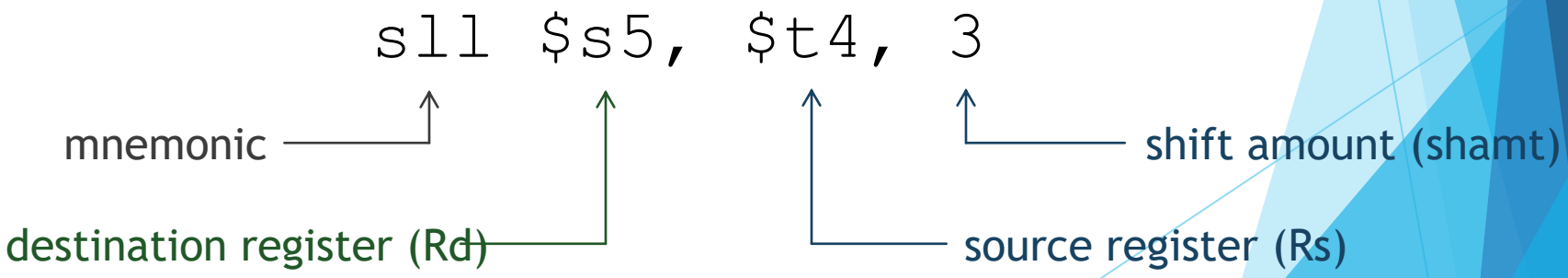
source register (Rs)

constant (immediate)

MIPS Shift Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
shift left logical	sll Rd, Rs, shamt	R[Rd] = R[Rs] << shamt	R
shift right logical	srl Rd, Rs, shamt	R[Rd] = R[Rs] >> shamt	R

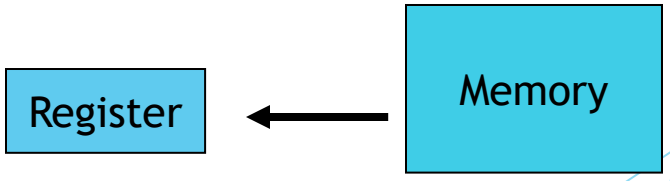
- Shift instructions use register operands only.
- Shift instructions use the *shift amount* (shamt) field.



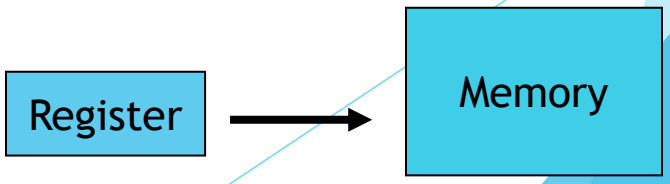
Memory Access Operations

- Values must be fetched from memory before instructions can perform operation on them (Fetch step in Instruction Cycle)
- Load instruction is responsible for reading an operand from memory
- Store instruction is responsible for writing an operand to memory
- General RISC operations include the following steps:
  - Load(s) to obtain operands from memory and bring them into registers
  - Operations performed only on values in target registers
  - Store(s) to update the memory by writing the result from register back to memory

Load word
lw $t0, memory-address



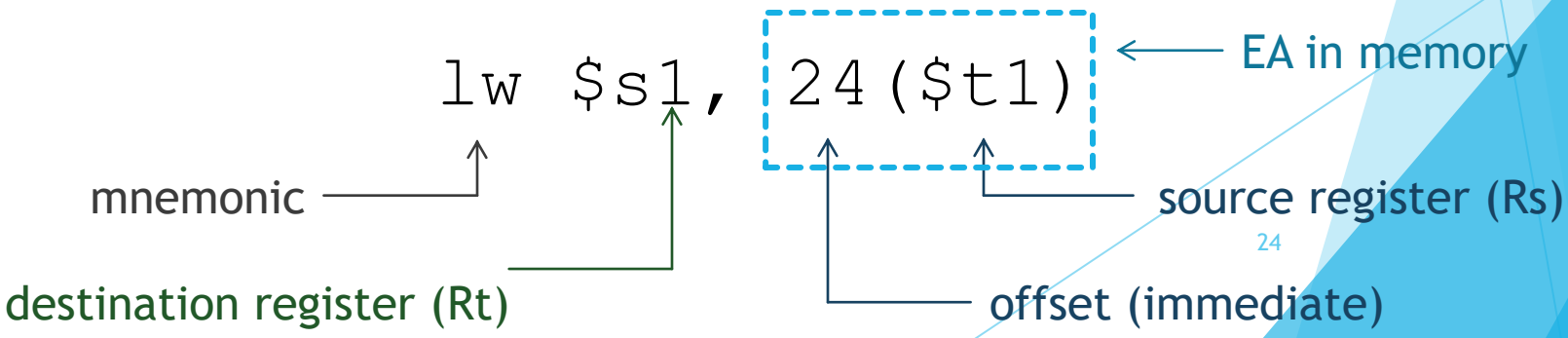
Store word
sw $t0, memory-address



MIPS Load Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
load word	lw Rt, imm.(Rs)	R[Rt] = M[R[Rs] + immediate]	I
load half word	lh Rt, imm.(Rs)	R[Rt] = M[R[Rs] + immediate]	I
load byte	lb Rt, imm.(Rs)	R[Rt] = M[R[Rs] + immediate]	I
load half unsigned	lhu Rt, imm.(Rs)	R[Rt] = M[R[Rs] + immediate]	I
load byte unsigned	lbu Rt, imm.(Rs)	R[Rt] = M[R[Rs] + immediate]	I

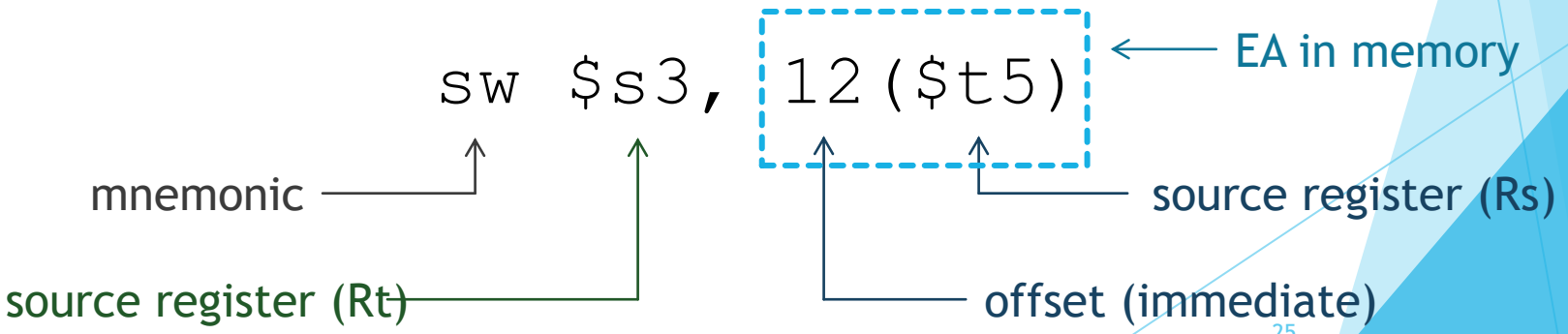
- Load instructions transfer data from memory to a register (read).
  - Load instructions use an *Effective Address* to access memory.
  - Load instructions use Rt as the **destination** register.



MIPS Store Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
store word	sw Rt, imm.(Rs)	M[R[Rs] + immediate] = R[Rt]	I
store half word	sh Rt, imm.(Rs)	M[R[Rs] + immediate] = R[Rt]	I
store byte	sb Rt, imm.(Rs)	M[R[Rs] + immediate] = R[Rt]	I

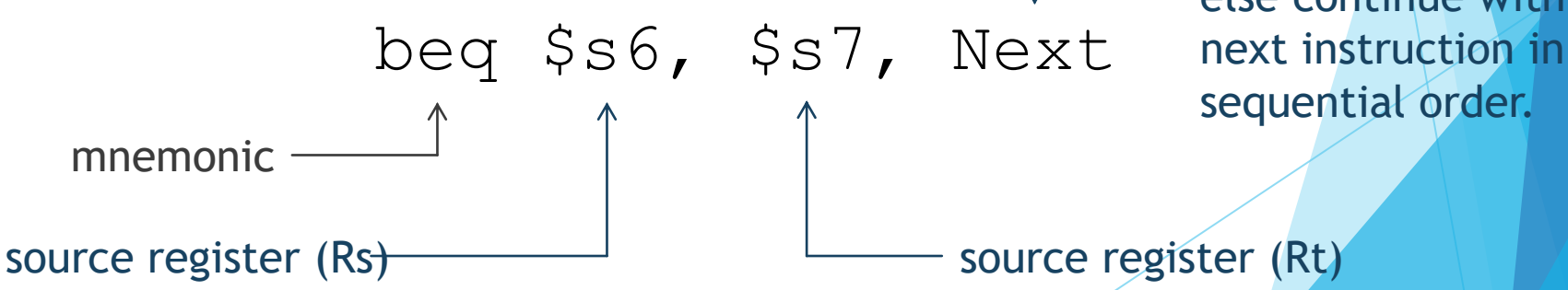
- Store instructions transfer data from a register to memory (write).
  - Store instructions use an *Effective Address* to access memory.
  - Store instructions use Rt as a **source** register.



MIPS Branch Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
branch on equal	beq Rs, Rt, label	If R[Rs] = R[Rt] goto *label*	I
branch on not equal	bne Rs, Rt, label	If R[Rs] != R[Rt] goto *label*	I

- Conditional branch instruction compares register operands.
  - Calculates the difference between the operands.
  - Uses the Zero flag from the ALU.
- Calculates the branch target address.



MIPS Branch Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
branch on equal	beq Rs, Rt, immediate	If R[Rs] = R[Rt] PC = PC + 4 + (imm. << 2)	I
branch on not equal	bne Rs, Rt, immediate	If R[Rs] != R[Rt] PC = PC + 4 + (imm. << 2)	I

```
beq $s6, $s7, Next
```

- Branch condition is evaluated.
  - If condition is **true**, PC = *address of labeled instruction*
  - else PC = *address of next instruction in sequential order*.

PC = BranchAddr

PC = PC + 4

- Branch instructions use PC-relative addressing.
  - BranchAddr = PC + 4 + (immediate << 2)

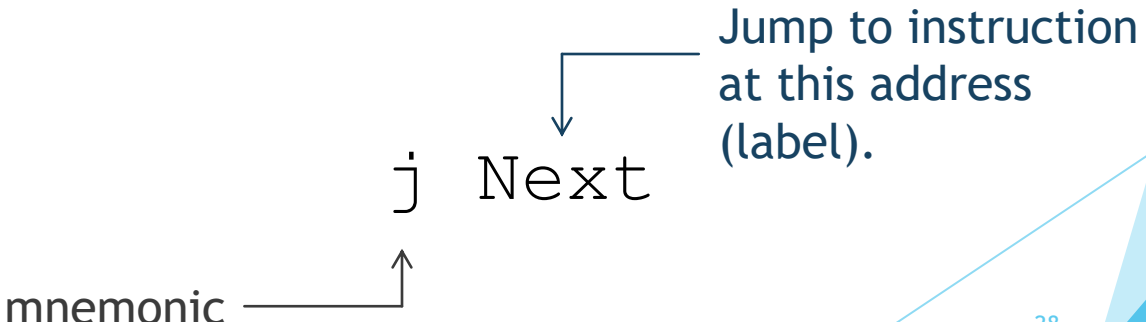
PC = Program Counter

Calculate branch target address using contents of PC

MIPS Jump Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
jump	j *label*	PC = address of instruction at *label*	J
jump register	jr Rs	PC = R[Rs]	R
jump and link	jal *label*	R[31] = PC + 4; PC = address if instruction at *label*	J

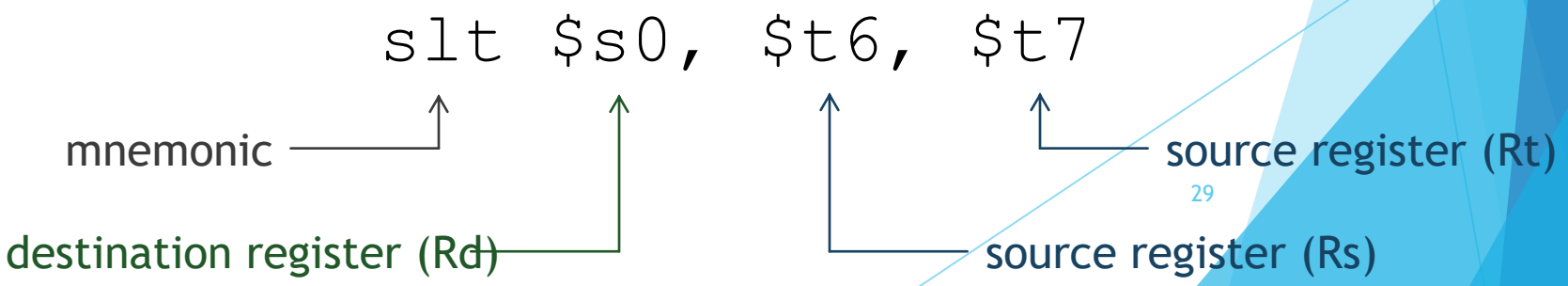
- *Jump* and *Jump and Link* calculate jump target address.
- *Jump Register* uses a register to specify jump target address.



MIPS Conditional Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
set on less than	slt Rd, Rs, Rt	If R[Rs] < R[Rt] then R[Rd] = 1, else R[Rd] = 0	R
set on less than unsigned	sltu Rd, Rs, Rt	If R[Rs] < R[Rt] then R[Rd] = 1, else R[Rd] = 0	R
set on less than immediate	slti Rt, Rs, immediate	If R[Rs] < SignExtImmediate then R[Rd] = 1, else R[Rd] = 0	I
set on less than immediate unsigned	sltiu Rt, Rs, immediate	If R[Rs] < SignExtImmediate then R[Rd] = 1, else R[Rd] = 0	I

- *Set On Less Than* instructions compare source operands.
  - "unsigned" instructions treat source operands as unsigned values.



MIPS Conditional Instructions

Description	Assembly Instruction	Register Transfer Notation	Format
set on less than	slt Rd, Rs, Rt	If R[Rs] < R[Rt] then R[Rd] = 1, else R[Rd] = 0	R
set on less than unsigned	sltu Rd, Rs, Rt	If R[Rs] < R[Rt] then R[Rd] = 1, else R[Rd] = 0	R
set on less than immediate	slti Rt, Rs, immediate	If R[Rs] < SignExtImmediate then R[Rt] = 1, else R[Rt] = 0	I
set on less than immediate unsigned	sltiu Rt, Rs, immediate	If R[Rs] < SignExtImmediate then R[Rt] = 1, else R[Rt] = 0	I

- *Set On Less Than* instructions compare source operands.
  - "unsigned" instructions treat source operands as unsigned values.

```
slti $s0, $t5, 0xC2C2
```

mnemonic

destination register (Rt)

source register (Rs)

constant (immediate)